Desarrollo Multiplataforma con React Native

Día 4: Consumo de APIs con fetch

Carlos Rodríguez Contreras
carlos.mailhub@gmail.com
karrocon

24 de abril de 2026

Conectar la app al mundo real

Agenda del día

- Repaso Día 3
- HTTP y JSON
- fetch + async/await
- useEffect para cargar datos
- Estados: loading / error / datos
- PokéAPI
- Diseño de la Pokédex
- Ejercicios
- Cierre

Objetivo del día

Cargar datos **reales desde internet** en una app.
Teoría breve → práctica inmediata.

Horario orientativo

16:00 Repaso
16:20 HTTP + JSON → mini-ej
17:00 fetch + async/await → mini-ej
18:00 **DESCANSO 30 min**
18:30 useEffect datos → mini-ej
19:00 Loading / error → mini-ej
19:30 PokéAPI → mini-ej
19:55 **Diseño Pokédex (clase)**
20:15 Ejercicios
20:25 Cierre 20:30 Fin

Repaso del Día 3

Navegación ya controlada

Repaso del Día 3

16:00 — 16:20 Repaso + preguntas

Lo que ya sabemos:

```
<NavigationContainer>
  <Stack.Navigator>
    <Stack.Screen
      name="Home"
      component={HomeScreen}
    />
    <Stack.Screen
      name="Detalle"
      component={DetalleScreen}
      options={({ route }) => ({
        title: route.params.nombre
      })}
    />
  </Stack.Navigator>
</NavigationContainer>

// Navegar:
navigation.navigate('Detalle',
  { nombre: 'Ana', rol: 'Dev' });
// Recibir params:
const { nombre, rol } = route.params;
```

Cubierto el Día 3:

- ✔ React Navigation instalado
- ✔ Stack Navigator + pantallas
- ✔ `navigation.navigate()` y `goBack()`
- ✔ Params tipados con `RootStackParamList`
- ✔ Opciones de cabecera
- ✔ Estructura `src/screens/` y `src/types/`

➔ Hoy empezamos

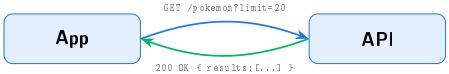
HTTP + JSON
`fetch` + `async/await`
`useEffect` para cargar datos
Estados: loading / error / datos
Primera llamada a PokéAPI

HTTP y JSON

Cómo hablan app y servidor

HTTP y JSON — conceptos clave

16:20 — 17:00 HTTP + JSON



Códigos de respuesta:

Cód.	Significado	Acción
200	OK	mostrar datos
404	No encontrado	mensaje error
429	Rate limit	reintentar
500	Error servidor	mensaje error

JSON: texto que representa objetos JS. `res.json()` lo convierte a objeto.

Respuesta JSON → tipo TypeScript:

```
// GET /users => array de objetos
[{ "id": 1, "name": "Leanne Graham",
   "email": "sincere@april.biz",
   "address": { "city": "Gwenborough" }
}]
```

```
type Usuario = {
  id: number;  name: string;
  email: string;
  address: { city: string };
};
```

3 pasos: abre URL → lee JSON → define `type`

⚠ Solo tipas los campos que vas a usar.

Mini-ejercicio — Inspeccionar una API

</> Mini-ejercicio (10 min)

Antes de escribir código, explora las APIs que usaremos hoy:

- Abre `https://jsonplaceholder.typicode.com/users` en el navegador
- Localiza los campos `name`, `email` y `address.city`
- Escribe el type `Usuario` en TypeScript que los describa
- Abre `https://pokeapi.co/api/v2/pokemon/1` y encuentra `name`, `weight`, `height` y `sprites.front_default`

Bonus: ¿qué estructura tiene `/pokemon?limit=5`? ¿Cuántos campos tiene cada resultado?

fetch + async/await

La forma moderna de hacer peticiones HTTP

¿Qué son las Promesas?

17:00 — 18:00 fetch + async/await

Promesa = operación que puede tardar

```
// Sin async/await (estilo antiguo):
fetch(URL)
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));

// Con async/await (estilo moderno):
async function cargar() {
  try {
    const res  = await fetch(URL);
    const data = await res.json();
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}
```

Analogía:

Una Promesa es como pedir comida en un restaurante:

- Haces el pedido (`fetch`)
- El camarero **promete** traerlo
- Puedes hacer otras cosas mientras
- Cuando llega → `await` desenvuelve el resultado

Para devs de Java

`async/await` ≈ `CompletableFuture.get()`

Pero sin bloquear el hilo UI.

fetch completo con comprobación de estado

```
type Usuario = {
  id: number; name: string;
  email: string;
  address: { city: string };
};

const URL = 'https://jsonplaceholder.typicode.
    com/users';

async function cargarUsuarios():
    Promise<Usuario[]> {
  const res = await fetch(URL);

  // fetch NO lanza error en 404/500:
  if (!res.ok) {
    throw new Error(
      `Error HTTP: ${res.status}`);
  }

  const data: Usuario[] = await res.json();
  return data;
}
```

Trampa común

`fetch` **no lanza error** si el servidor responde 404 o 500.

Solo lanza si hay error de red.

✔ Siempre comprueba `res.ok` o `res.status`.

Dos `await` necesarios:

1. `await fetch(URL)` — espera la cabecera HTTP
2. `await res.json()` — espera el cuerpo completo

`Promise<Usuario[]>` tipa explícitamente lo que devuelve la función asíncrona.

try / catch / finally — manejo de errores

```
async function cargarDatos():
    Promise<void> {
  try {
    setCargando(true);
    setError(null);

    const res = await fetch(URL);
    if (!res.ok) throw new Error(
      `Error ${res.status}`);

    const data: Usuario[] =
      await res.json();
    setDatos(data);

  } catch (e) {
    // e es 'unknown' en TS strict:
    setError((e as Error).message);

  } finally {
    // Se ejecuta SIEMPRE:
    setCargando(false);
  }
```

try/catch/finally

try — código que puede fallar

catch — qué hacer si falla

finally — se ejecuta siempre (éxito o fallo)

⚠ En TypeScript estricto e es `unknown`. Usa `(e as Error).message` para leerlo.

✔ `finally` es perfecto para `setCargando(false)` — siempre se ejecuta.

Mini-ejercicio — fetch manual

</> Mini-ejercicio (15 min)

En `snack.expo.dev` o en tu proyecto:

- Escribe una función `cargarUsuarios()` con `async/await`
- Llama a `jsonplaceholder.typicode.com/users`
- Comprueba `res.ok` y lanza error si falla
- Llama a `cargarUsuarios()` con `.then(console.log).catch(console.error)`
- Cambia la URL a `/users/999` para ver el error 404

Bonus: usa `try/catch/finally` en lugar de `.then/.catch`.

Pista: `if (!res.ok) throw new Error(`Error: ${res.status}`)`

Descanso — 30 minutos

18:00 — 18:30 Descanso

30 minutos

Al volver: useEffect + estados de carga

useEffect para cargar datos

El hook que conecta componentes con el exterior

useEffect — cuándo ejecutar código

18:30 — 19:00 useEffect

Sintaxis:

```
useEffect(() => {
  // c digo a ejecutar
}, [dependencias]);
```

3 variantes:

```
// 1. Solo al montar:
useEffect(() => { cargarDatos(); }, []);

// 2. Cuando cambia 'id':
useEffect(() => { cargarDetalle(id); }, [id]);

// 3. En cada render (sin array):
useEffect(() => { console.log('render'); });
```

¿Por qué no en el cuerpo?

El cuerpo se ejecuta en cada render.

Sin `useEffect`, haríamos fetch en cada pulsación.

`useEffect(fn, [])` → solo al montar.

No pongas `async` directo en `useEffect`. Usa función interna:

```
useEffect(() => {
  async function load() { ... }
  load();
}, []);
```

Estados: loading / error / datos

El patrón que usaremos en toda la Pokédex

El patrón completo: 3 estados

19:00 — 19:30 Loading / error / datos

```
export default function App() {
  const [datos, setDatos] = useState<Usuario[]>([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState<string|null>(null);
  useEffect(() => { cargar(); }, []);
  async function cargar(): Promise<void> {
    try {
      setCargando(true);  setError(null);
      const res = await fetch(URL);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setDatos(await res.json());
    } catch (e) {
      setError((e as Error).message);
    } finally { setCargando(false); }
  }
  if (cargando) return (
    <ActivityIndicator size="large"
      style={{ flex: 1 }} color="#2078F3" />);
  if (error) return (
    <View style={s.centrado}>
      <Text style={s.error}>{error}</Text>
      <Button title="Reintentar" onPress={cargar} />
    </View>
  );
  return (
    <FlatList data={datos}
      keyExtractor={u => String(u.id)}
      renderItem={({ item }) => (
        <Text>{item.name}     {item.email}</Text>
      )} />
  );
}
```

3 estados

cargando: **true** → spinner
error: **string** → mensaje + reintentar
datos: **array** → lista

Por qué `string | null`

`null` = no hay error.
`string` = mensaje del error.
TypeScript obliga a comprobar si es `null` antes de mostrar.

⚠ Reinicia `setError(null)` al inicio de cada carga para no mostrar el error antiguo.

Mini-ejercicio — Usuarios con 3 estados

</> Mini-ejercicio (15 min)

Implementa el patrón completo con datos reales:

- Declara los 3 estados: `datos`, `cargando`, `error`
- Llama a `jsonplaceholder.typicode.com/users` en `useEffect`
- Muestra `ActivityIndicator` mientras carga
- Muestra el error + botón «Reintentar» si falla
- Muestra la lista de usuarios (nombre + email + ciudad) si ok

Bonus: añade `RefreshControl` para recargar tirando hacia abajo.

Simular error: cambia la URL a `/users/9999` para ver el 404

PokéAPI

La API que usaremos toda la semana

PokéAPI — estructura de datos

19:30 — 19:55 PokéAPI

GET /pokemon?limit=20

```
{
  "count": 1302,
  "results": [
    { "name": "bulbasaur", "url": ".../1/" },
    { "name": "ivysaur",   "url": ".../2/" },
    ...
  ]
}
```

Tipo TypeScript:

```
type PokemonLista = {
  name: string;  url: string;
  id:   string;  // extraído de url
};
const getId = (url: string): string =>
  url.split('/').filter(Boolean).pop()!;
// Sprite oficial (GitHub raw):
const SPRITES =
  'https://raw.githubusercontent.com/' +
  'PokeAPI/sprites/master/sprites/pokemon';
const sprite = (id: string) =>
  `${SPRITES}/${id}.png`;
```

Endpoints que usaremos:

Endpoint	Usa
/pokemon?limit=N	Día 4-5
/pokemon/{id}	Días 5-6

Sin API key

PokéAPI es 100 % gratuita y sin registro. Límite: ~100 req/min por IP (más que suficiente en desarrollo).

pokeapi.co/docs/v2

Adaptar el patrón a PokéAPI

```
const API = 'https://pokeapi.co/api/v2';
const getId = (url: string) =>
  url.split('/').filter(Boolean).pop()!;
const cap = (s: string) =>
  s[0].toUpperCase() + s.slice(1);
const SPRITES =
  'https://raw.githubusercontent.com/' +
  'PokeAPI/sprites/master/sprites/pokemon';
const sprite = (id: string) =>
  `${SPRITES}/${id}.png`;

async function cargarPokemon(): Promise<void> {
  const res = await fetch(`${API}/pokemon?limit=20`);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const json = await res.json() as {
    results: Array<{ name: string; url: string }>;
  };
  setDatos(json.results.map(p => ({
    name: p.name, url: p.url, id: getId(p.url),
  })));
}
```

FlatList con imagen:

```
<FlatList
  data={datos}
  numColumns={2}
  keyExtractor={p => p.id}
  renderItem={({ item }) => (
    <View style={s.card}>
      <Image
        source={{ uri: sprite(item.id) }}
        style={{ width: 90, height: 90 }}
        resizeMode="contain"
      />
      <Text>#{item.id.padStart(3,'0')}</
          Text>
      <Text>{cap(item.name)}</Text>
    </View>
  )}
/>
```

✔ Las imágenes de Pokémon son PNG alojados en GitHub: carga rápida y sin caché propia.

Mini-ejercicio — Lista de Pokémon

</> Mini-ejercicio (15 min)

Adapta el ejercicio de usuarios a PokéAPI:

- Cambia el tipo a `PokemonLista` y la URL a `/pokemon?limit=20`
- Transforma `json.results` con `getId()`
- Muestra el sprite con `<Image source={{ uri: sprite(item.id) }} />`
- Usa `numColumns={2}` en la FlatList

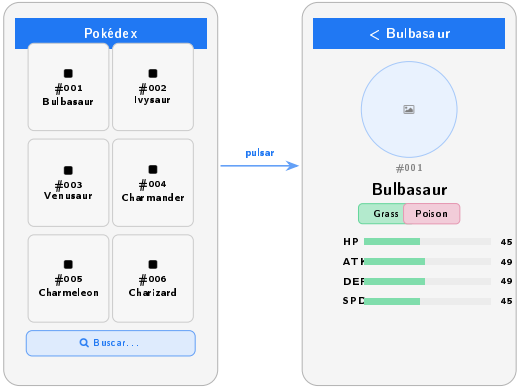
Bonus: al pulsar un Pokémon, muestra su peso y altura en un `Modal` (fetch a `/pokemon/{id}`).

Diseñamos la Pokédex juntos

Pensar la app antes de escribir código

¿Qué app vamos a construir?

19:55 — 20:10 Diseño colaborativo — entre todos



Pantalla 1: Lista

Grid de tarjetas

Pantalla 2: Detalle

Sprite + nombre + tipo + stats

Analizamos la app — debate en clase

Pantallas

- **ListaScreen** — grid de Pokémon
- **DetalleScreen** — info de uno

Componentes en ListaScreen

- `PokemonCard` — imagen, número, nombre, tipo
- `SearchBar` — input de búsqueda
- `LoadingSpinner` / `ErrorView`

Componentes en DetalleScreen

- `SpriteImage` — imagen grande
- `TypeBadge` — pastilla de tipo (color)
- `StatBar` — barra de estadística

¿Dónde necesitamos estado?

- `pokemon[]` — lista cargada de la API
- `cargando: boolean` — spinner
- `error: string|null` — mensaje de error
- `busqueda: string` — filtro del SearchBar

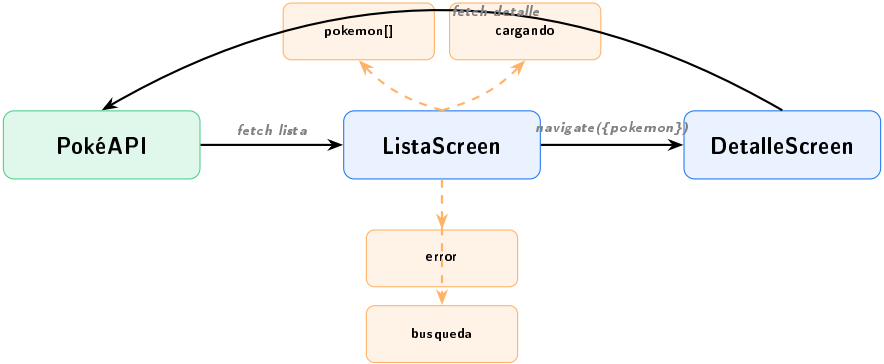
Navegación

- Stack Navigator: Lista → Detalle
- Parámetro: `{ pokemon: PokemonData }`

? ¿Quién os manda este debate?

El profesor pregunta primero
— los alumnos responden.

Flujo de datos de la app



Lista: carga `/pokemon?limit=151`, guarda en estado,
pasa el objeto completo a Detalle por params.

Detalle: recibe `PokemonData` por params y opcionalmente hace un segundo fetch a `/pokemon/{id}` para los stats.

Parada: Ejercicios en clase

19:55 — 20:25 Trabajo autónomo guiado

Los alumnos trabajan en los 2 ejercicios.

El profesor:

- Rota por los puestos
- Ayuda con errores de `async/await`
- Error frecuente: olvidar `res.ok` o los dos `await`
- Muestra solución si hay bloqueo general

Ejercicios en: `dia-04/ejercicios/`

Criterio de éxito

- **Ej1:** usuarios con loading, error y recargar
- **Ej2:** grid Pokémon con sprites

Bonus: Modal con peso y altura

Error más común:
`fetch` no lanza en 404.
→ recuerda comprobar `res.ok`

Resumen del día

Hoy hemos aprendido:

- ✔ Peticiones HTTP con **fetch**
- ✔ **async** / **await** y promesas
- ✔ **try** / **catch** / **finally**
- ✔ **useEffect** para cargar al montar
- ✔ Patrón **loading** / **error** / **datos**
- ✔ Tipos TypeScript para JSON
- ✔ PokéAPI: endpoints y utilidades

Días 5-6 — Próxima sesión

- **Pokédex completa** de verdad
- Arquitectura `src/` modular
- Componente `PokemonCard` reutilizable
- Pantalla de detalle con stats y tipos
- Los 151 Pokémon originales

➜ Aplicamos TODO lo del curso en un proyecto real

Recursos y referencias

Documentación oficial:

- `MDN - Fetch API`
- `MDN - async/await`
- `react.dev - useEffect`
- `pokeapi.co/docs/v2`

APIs de prueba:

- `jsonplaceholder.typicode.com`
- `reqres.in` — simula CRUD

Usa **Snack** para probar fetch rápidamente sin instalar nada.

`karrocon/taller-react-native-fp`
Ejercicios y soluciones del curso

¿Preguntas?

Antes de salir:

✔ Sabes hacer `fetch` con `async/await`
✔ Manejas los 3 estados: loading, error, datos
✔ Sabes usar `useEffect` con array vacío

Si no es así → **ahora es el momento de preguntar**